UAV Software Engineering: Build Systems

CMake and Out-of-Source Builds

1 Theoretical Foundation: The Need for Build Systems

In large-scale robotics projects like PX4 firmware or ROS 2 workspaces, software is composed of thousands of source files, external libraries, and cross-dependencies. Compiling these ecosystems manually via terminal commands (e.g., `g++ main.cpp -o node`) is unmaintainable.

To solve this, the industry standard is **CMake**. CMake is not a compiler itself; rather, it is a cross-platform build system generator. It reads high-level configuration rules and automatically generates the low-level build files (such as Makefiles) required to compile the project efficiently.

2 UAV System Application: Configuring `CMakeLists.txt`

The core of CMake configuration is a text file strictly named `CMakeLists.txt`. To build a multi-threaded telemetry node, the minimal configuration requires defining the project, adding the executable, and linking necessary libraries:

- **Basic Setup**: `cmake_minimum_required(VERSION 3.10)` and `project(telemetry_node)` define the baseline.
- **Compilation Target**: `add_executable(telemetry_node main.cpp)` instructs the system to compile the source code into an executable file.
- **Dependency Linking**: While C++ Standard Template Library (STL) headers like `<queue>` or `<mutex>` are natively included by the compiler, external OS-level libraries are not. To use hardware threads on Linux, we must explicitly find and link the system's thread library using `find_package(Threads REQUIRED)` and `target_link_libraries(telemetry_node Threads::Threads)`.

3 Implementation Details: Out-of-Source Builds

In professional C++ development, build artifacts (Makefiles, object files, binaries, and cache files) must never pollute the source code directory. This is solved by utilizing an **out-of-source build** strategy:

1. Create an isolated directory: `mkdir build` and `cd build`.
2. Generate build files targeting the parent directory: `cmake ..`
3. Compile the binary: `make`
4. Execute the resulting program locally: `./telemetry_node`

To maintain a clean version control history in Git, the `build/` directory must be explicitly added to the `.gitignore` file. This ensures that machine-specific compiled binaries and temporary artifacts are never committed to the remote repository.